Sorting Visualizer

Nour El Atawy : 23010925

March 2026

1 System Structure & Design Decisions

With the design of this application, i tried my best not to stray far from the clean code principles and the SOLID principles in order to make reasoning about the system and modularizing it a more manageable task

1.1 The Algorithms

For the algorithms i tried making them as decoupled as possible from the visualization logic, and any logic that simply shouldn't pollute the main methods.

1.1.1 Base class

```java
public abstract class SortingStrategy<E extends Comparable<E>>{

    void addToMetrics(SortingStep<E> step, SortingMetrics<E> metrics) {
        if (metrics != null){
            metrics.addStep(step);
        }
    }
    public abstract List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics);

    public List<E> sort(List<E> list, Comparator<E> comparator){
        return sort(list, comparator, null);
    }

    public List<E> sort(List<E> list){
        return sort(list, Comparator.naturalOrder());
    }
}
```

Figure 1: Strategy Base Class

I thought about making all the sorting algorithms extend some abstract class that define the main method of all sorting algorithms (with some convenient overloads). i tried avoiding restricting or highly specific data types so as you can see here, i went with the Generic List of Java.util, but with a condition that the Generic is implementing the Comparable interface which is self-explanatory.

i also made the more general overload for sort take in an object of type SortingMetric this object should be responsible for storing the steps of the sorting process, and some statistics and information about the sorting process like the number of swaps, comparisons ..etc i went with such a design because it tries to balance between the ability to precisely mark a step and store it without making the SortingStrategy stateful or forcing it to keep track of these metrics itself which violates the Single Responsibility Principle and between having a cleaner more decoupled design.

1.1.2 $O(n^2)$ Algorithms

```java
public class BubbleSort<E extends Comparable<E>> extends SortingStrategy<E>{

    public List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {
        for (int i = 0; i < list.size(); i++){
            for (int j = 0; j < list.size() - 1; j++){
                super.addToMetrics(new CompareStep<>(j, j + 1), metrics);
                if (comparator.compare(list.get(j + 1),list.get(j)) < 0){
                    Collections.swap(list, j, j + 1);
                    super.addToMetrics(new SwapStep<>(j, j + 1), metrics);
                }
            }
        }
        return list;
    }
}
```

Figure 2: Bubble Sort

```java
public class InsertionSort<E extends Comparable<E>> extends SortingStrategy<E> {

    public List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {
        for (int i = 1; i < list.size(); i++) {
            E pivot = list.get(i);
            int j = i - 1;
            super.addToMetrics(new CompareStep<>(i, j), metrics); // initial comparison
            while (j >=0 && comparator.compare(list.get(j), pivot) > 0) {
                list.set(j+1, list.get(j));
                super.addToMetrics(new CompareStep<>(i, j), metrics);
                super.addToMetrics(new SwapStep<>(j, j + 1), metrics);
                j--;
                // we technically didn't swap but the behavior is the same
            }
            if (j != i - 1) {
                list.set(j+1, pivot);
                super.addToMetrics(new SetStep<>(j + 1, pivot), metrics);
            }
        }
        return list;
    }

}
```

Figure 3: Insertion Sort

```java
public class SelectionSort<E extends Comparable<E>> extends SortingStrategy<E> {

    public List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {

        for (int i = 0; i < list.size(); i++) {
            int bestIndex = i;
            for (int j = i + 1; j < list.size(); j++) {
                super.addToMetrics(new CompareStep<>(j, bestIndex), metrics);
                if (comparator.compare(list.get(j), list.get(bestIndex)) < 0) {
                    bestIndex = j;
                }
            }
            if (bestIndex != i) {
                Collections.swap(list, bestIndex, i);
                super.addToMetrics(new SwapStep<>(bestIndex, i), metrics);
            }
        }
        return list;
    }


}
```

Figure 4: Selection Sort

in these algorithms what i was previously describing is applied here, the algorithm mainly cares about "actually" sorting the array, not much else, but when an operation happens, it just tells the metrics to keep track of it and that's actually the only point where the sorting logic and the metric handling logic meet and then they part afterwards. so they are not perfectly decoupled, however it can give us more control over where the metrics is asked to keep track of some step (an alternative approach could have been that i define another listClass that provides the necessary operations for sorting and on invocation of any of it's methods it keeps track of some step which would be nice however it would make me lose the flexibility of using a generic List).

1.1.3 Recursive Algorithms

```java
public class MergeSort<E extends Comparable<E>> extends SortingStrategy<E> {


    private List<E> merge(List<E> list, int start1, int start2, int end,
                          Comparator<E> comparator, SortingMetrics<E> metrics) {
        List<E> merged = new ArrayList<>();
        int itr1 = start1;
        int itr2 = start2;
        while (itr1 < start2 && itr2 < end) {

            super.addToMetrics(new CompareStep<>(itr1, itr2), metrics);
            if (comparator.compare(list.get(itr1), list.get(itr2)) < 0) {
                merged.add(list.get(itr1));
                itr1++;
            }
            else {
                merged.add(list.get(itr2));
                itr2++;
            }
        }
        while (itr1 < start2) {
            merged.add(list.get(itr1));
            itr1++;
        }
        while (itr2 < end) {
            merged.add(list.get(itr2));
            itr2++;
        }
        return merged;
    }

    public List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {
        mergesort(list, start: 0, list.size(), comparator, metrics);
        return list;
    }

    private void mergesort(List<E> list, int start, int end, Comparator<E> comparator, SortingMetrics<E>
        if (list == null || start >= end - 1) return;

        int mid = (start + end) / 2;
        mergesort(list, start, mid, comparator, metrics);
        mergesort(list, mid, end, comparator, metrics);
        List<E> merged = merge(list, start, mid, end, comparator, metrics);
        // instead of only returning the merged array we write it back in the original array before retur
        // to mimic the behavior of sorting in place and have all the changes reflected in the original a

        for (int i = start; i < end; i++) {
            list.set(i, merged.get(i - start));
            super.addToMetrics(new SetStep<>(i, merged.get(i - start)), metrics);
        }
    }

}
```

Figure 5: Merge Sort

Here i had to do a twist on the classical merge sort; that is, i rewrite the result of the merge of the left and right halves back into the original array, and i use start + end instead of subLists because that way i avoid all the issues caused from Global indexing vs Local Indexing, one of which is the inability to provide the correct step information that will be later used for visualization, and many other issues, so abit more complex indeed, however, it ensures correctness in cases where subListing would be problematic.

```java
public class QuickSort<E extends Comparable<E>> extends SortingStrategy<E>{

    private int partition(List<E> list, int start, int end,
                          Comparator<E> comparator, SortingMetrics<E> metrics) {
        E pivot = list.get(end - 1);
        int lastSE = start - 1; //index to the last element that is not larger than the pivot
        int itr = start;
        while(itr < end - 1) {
            super.addToMetrics(new CompareStep<>(itr, end - 1), metrics);
            if(comparator.compare(list.get(itr),pivot) <= 0){
                lastSE += 1;
                Collections.swap(list, itr, lastSE);
                super.addToMetrics(new SwapStep<>(itr, lastSE), metrics);
            }
            itr += 1;
        }

        Collections.swap(list, lastSE + 1, end - 1); //position the pivot in its correct spot
        super.addToMetrics(new SwapStep<>(lastSE + 1, end - 1), metrics);
        return lastSE + 1;
    }

    /*
    This method exploits the principle of randomized algorithms to avoid worst-case scenarios
    and get the O(n.lgn) average running time even in Sorted or Reversely Sorted situations
     */
    private int randomizedPartition(List<E> list, int start, int end,…


    public List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {
        quickSort(list, start: 0, list.size(), comparator, metrics);
        return list;
    }

    private void quickSort(List<E> list, int start, int end,
                           Comparator<E> comparator,  SortingMetrics<E> metrics) {
        if (list == null || start >= end - 1) {
            return;
        }
        int pivot_idx = randomizedPartition(list,start,end, comparator,  metrics);
        quickSort(list, start, pivot_idx, comparator, metrics);
        quickSort(list, pivot_idx + 1, end, comparator, metrics);
        // since we are passing a sublist then we are passing a view to the same array
        // and the partition method operates on the original array itself

        // so it utilizes the sorting-in-place feature of QuickSort and we simply
    }


}
```

Figure 6: Quick Sort

for the Quick Sort i resolved to the randomized version that is known to significantly reduce the likelihood of getting the worst-case Quadratic running time of Quick Sort, and the issue we encountered with the Merge Sort is absent here due to it's Sort-in-place nature.

```java
public class HeapSort<E extends Comparable<E>> extends SortingStrategy<E> {

    private void heapify(List<E> list, Comparator<E> comparator, int index, SortingMetrics<E> metrics) {
        int leftChild = index * 2;
        int rightChild = index * 2 + 1;

        int maxIdx = index;

        if (leftChild < list.size()) {
            super.addToMetrics(new CompareStep<>(leftChild, maxIdx), metrics);
            if(comparator.compare(list.get(leftChild), list.get(maxIdx)) > 0) {
                maxIdx = leftChild;
            }
        }
        if (rightChild < list.size()) {
            super.addToMetrics(new CompareStep<>(rightChild, maxIdx), metrics);
            if(comparator.compare(list.get(rightChild), list.get(maxIdx)) > 0) {
                maxIdx = rightChild;
            }
        }

        if (maxIdx != index) {
            Collections.swap(list, maxIdx, index);
            super.addToMetrics(new SwapStep<>(maxIdx, index), metrics);
            heapify(list, comparator, maxIdx, metrics);
        }
    }

    private void buildHeap(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {
        for (int i = list.size() / 2; i >= 0; i--) {
            heapify(list, comparator, i, metrics);
        }
    }

    @Override
    public List<E> sort(List<E> list, Comparator<E> comparator, SortingMetrics<E> metrics) {
        buildHeap(list, comparator,  metrics);

        for (int i = list.size() - 1; i >= 0; i--) {
            Collections.swap(list,0, i);
            super.addToMetrics(new SwapStep<>(firstIdx: 0, i), metrics);
            heapify(list.subList(0,i), comparator, index: 0,  metrics);
        }
        return list;
    }


}
```

Figure 7: Heap Sort

I followed the known textbook implementation of Heap Sort, going from the lowest non-leaf nodes maintaining the heap property and going up the heap to construct our max heap and then keep extracting the max element (the root), swapping it with the last elements of the heap and calling heapify again to restore the Heap property.

1.2 The Metrics

1.2.1 SortingMetrics

```java
public class SortingMetrics<E> {
    private int comparisonCount;
    private int setCount;
    private int swapCount;
    private final List<SortingStep<E>> steps;

    public SortingMetrics() {
        steps = new ArrayList<SortingStep<E>>();
    }

    public void addStep(SortingStep<E> step) {
        if (step instanceof CompareStep<E>) {
            comparisonCount++;
        }
        else if (step instanceof SetStep<E>) {
            setCount++;
        }
        else if (step instanceof SwapStep<E>) {
            swapCount++;
        }
        steps.add(step);
    }

    public int getComparisonCount() {
        return comparisonCount;
    }

    public int getSwapCount() {
        return swapCount;
    }

    public int getSetCount() {
        return setCount;
    }

    public List<SortingStep<E>> getSteps() {
        return Collections.unmodifiableList(steps);
    }
}
```

Figure 8: Sorting Metrics Handler Class

This class is the one responsible for tracking all the intermediate steps of any sorting process made by any SortingStrategy as we discussed earlier,its structure is as simple as its role (simple however critical).

1.2.2 Steps

```java
public interface SortingStep<E> {

    public void visualizeOn(VisualizationBars<E> visualizationBars);

}
```

Figure 9: Sorting Step Interface

This simple interface draws attention to a very important thing regarding the SortingSteps, that the step knows how it should be visualized, it doesn't care what array from which it was generated, and the SortingStrategy doesn't have any idea on how it will be handled or visualized, the step is the object that knows this information which is basically a form of a Command Design Pattern. and that signals a clear -enough seperation of concerns. (No need to dive deep into the internals of this interface's concretions).

1.3 The GUI

1.3.1 Bar Visualization Handler

```java
public void addBar(Rectangle bar, T value) {  2 usages  nelatawy
    int intVal = mappingFunction.applyAsInt(value);

    if (intVal > max.get()) {
        max.set(intVal);
    }
    else if (intVal < min.get()) {
        min.set(intVal);
    }
    DoubleProperty height = new SimpleDoubleProperty(intVal);

    DoubleBinding baseBinding = min.subtract( i: 1);
    DoubleBinding maxBinding = max.subtract(min).add(1);
    DoubleBinding binding = height.subtract(baseBinding).divide(maxBinding).multiply(containerHeight);

    bar.heightProperty().bind(binding);
    bars.add(bar);
    values.add(value);
    heights.add(height);
}

public void updateHeightAt(int index, T value) {  3 usages  nelatawy
    values.set(index, value);
    heights.get(index).set(mappingFunction.applyAsInt(value));
}

public void markBarAt(int idx, Label label) {  7 usages  nelatawy
    Rectangle target = bars.get(idx);
    target.getStyleClass().removeAll( ...es: "bar-sorted", "bar-highlight", "bar-focus");

    switch (label) {
        case SORTED:
            target.getStyleClass().add("bar-sorted");
            break;
        case HIGHLIGHT:
            target.getStyleClass().add("bar-highlight");
            break;
        case FOCUS:
            target.getStyleClass().add("bar-focus");
            break;
        case NONE:
            break;
    }
}

public void resetStyling() {  1 usage  nelatawy
    for (int i = 0; i < bars.size(); i++) {
        markBarAt(i, Label.NONE);
    }
}
```

Figure 10: VisualizationBars

This class relies heavily on the feature of Binding Properties provided by JavaFX, it handles the current values that each bar represents, which is seperate from the height of each bar, this seperation might allow the visualization of any type that allows enumeration not just objects of the Integer class. In addition, it also keeps track of the actual Rectangle objects that the renderer draws, which means it also has access to styling properties which makes adding css class styles to a bar depending on it's status (FOCUSED, HIGHLIGHTED, NONE) possible. This class also does a sequence of operations to create the binding that will be used as the height of the bars, these operations aim to provide the ability to visualize negative numbers as well as normalize the heights to be proportional to both the scale of the numbers and the container height.

1.3.2 The Controllers

```java
@Override  nelatawy
public void initialize(URL url, ResourceBundle resourceBundle) {...}

// generates an array and visualizes it
@FXML  1 usage  nelatawy
public void genVisArray() throws IOException {
    generateArray();
    if (arr.size() > 250) {
        ArrayVisManager.getInstance().loadSnapshot(arr, visualizationBars); //adding them as is
        goToVisualizeBtn.setDisable(true); // prevent user from trying to visualize it
        return;
    }
    goToVisualizeBtn.setDisable(false);
    linkToVisBars();

}

public void generateArray() throws IOException {...}

private void linkToVisBars() {  1 usage  nelatawy
    barContainer.setAlignment(Pos.BOTTOM_LEFT);
    for (int i = 0; i < arr.size(); i++) {
        Rectangle rectangle = new Rectangle();
        rectangle.setWidth(barContainer.getPrefWidth() / arr.size());
        rectangle.setFill(Color.BLUE);
        rectangle.getStyleClass().add("bar");
        visualizationBars.addBar(rectangle, arr.get(i));
        barContainer.getChildren().add(rectangle);
    }
    ArrayVisManager.getInstance().loadSnapshot(arr, visualizationBars);
}


@FXML  nelatawy
private void goToVisualizer(ActionEvent event) throws IOException {
    CommonUtils.goTo(event,  fxmlName: "visualizer");
}

@FXML  nelatawy
private void goToComparison(ActionEvent event) throws IOException {
    CommonUtils.goTo(event,  fxmlName: "comparison");
}

@FXML  nelatawy
private void onModeSelected(ActionEvent event) throws IOException {...}
```

Figure 11: ArrayGenController

ArrayGenController This controller handles the UI page responsible for generating the array and saving it to the system for the visualizer and/or the comparator to later operate on. It provides the basic functions that is needed from the UI, like generating the array based on the selected mode (RANDOM, SORTED, REVERSED, FROMFILE) and visualizing it (if the number of elements is small enough) adding the bar to VisBars tracker and providing some

basic navigation functionality.

```
private void reset(){...}

@FXML  nelatawy
private void pause(){...}

@FXML  nelatawy
private void resume(){...}


@FXML  1 usage  nelatawy
public void visualizeSort() {
    if (isVisualizing) {
        return;
    }
    if (priorlySorted) {
        reset();
    }
    // to allow for resets
    SortingStrategy<Integer> sortingAlgorithm = CommonUtils.getSortingAlgorithm(algorithmSelector.getValue());
    sortingAlgorithm.sort(arr, Comparator.naturalOrder(), metrics);
    priorlySorted = true;
    new Thread(() -> {
        visualize(metrics.getSteps());
    }).start();

}

public void visualize(List<SortingStep<Integer>> steps) {  nelatawy
    isVisualizing = true;

    for (int i = 0; i < steps.size(); i++) {
        int idx = i;
        KeyFrame keyFrame = new KeyFrame(Duration.millis( v: 1000/24.0 * (i + 1)), ActionEvent e -> {
            visualizationBars.resetStyling();
            steps.get(idx).visualizeOn(visualizationBars);
        });
        timeline.getKeyFrames().add(keyFrame);
    }
    KeyFrame sortedKF = new KeyFrame(Duration.millis( v: 1000/24.0 * (steps.size() + 1)), ActionEvent e -> {
        visualizationBars.markAllSorted();
    });
    timeline.getKeyFrames().add(sortedKF);
    timeline.setOnFinished( ActionEvent e -> {
        isVisualizing = false;
        showStats();
    });
    timeline.play();
}

void showStats(){...}
@FXML  nelatawy
private void goToArrayGen(ActionEvent event) throws IOException {...}
```

Figure 12: SortingController

SortingController This controller handles the Visualization tab, that handles the sorting and visualization of any of the supported algorihms, providing necessary like sorting the array and redrawing the bars if it was previously visualized by loading the snapshot that was stored at the moment of generation

by the ArrayGenController, Notice that : the VisualizationBars class provides a constructor that takes in another VisualizationBars object, this performs a deep copy so on resetting we load a snapshot to make sure we don't make changes that get reflected in the original array. Afterwards the Controller creates a new Thread and it runs the sorting and visualizing within it so that it doesn't block the main Thread which is used by JavFX for rendering, which might cause it to hang.

```
static class SortStat {  11 usages   nelatawy

    final StringProperty algorithm = new SimpleStringProperty();  3 usages
    final IntegerProperty size = new SimpleIntegerProperty();  3 usages
    final LongProperty comparisons = new SimpleLongProperty();  3 usages
    final LongProperty swaps = new SimpleLongProperty();  3 usages
    final LongProperty writes = new SimpleLongProperty();  3 usages
    final DoubleProperty minRuntime = new SimpleDoubleProperty();  3 usages
    final DoubleProperty maxRuntime = new SimpleDoubleProperty();  3 usages
    final DoubleProperty meanRuntime = new SimpleDoubleProperty();  3 usages

    public SortStat(String algorithm,int size,  1 usage   nelatawy
                    long comparisons, long swaps, long writes,
                    double minRuntime, double maxRuntime, double meanRuntime) {
        this.algorithm.set(algorithm);
        this.size.set(size);
        this.comparisons.set(comparisons);
        this.swaps.set(swaps);
        this.writes.set(writes);
        this.minRuntime.set(minRuntime);
        this.maxRuntime.set(maxRuntime);
        this.meanRuntime.set(meanRuntime);
    }
    public String toString() {   nelatawy
        return  this.algorithm.get() + "," +
                this.size.get() + "," +
                this.comparisons.get() + "," +
                this.swaps.get() + "," +
                this.writes.get() + "," +
                this.minRuntime.get() + "," +
                this.maxRuntime.get() + "," +
                this.meanRuntime.get();
    }

}
```

Figure 13: SortStats

SortStats This class is a POJO that contains all the data fields needed to make up a record in the StatsTable.

```java
@Override  nelatawy
public void initialize(URL url, ResourceBundle resourceBundle) {
    for(SortingAlgorithm algorithm : SortingAlgorithm.values()){
        CheckBox checkBox = new CheckBox(algorithm.toString());
        algorithmCheckboxes.getChildren().add(checkBox);
    }
    arr = ArrayVisManager.getInstance().getArraySnapshot(); //already generated from ArrayGenerator

    runCount.setTextFormatter(CommonUtils.createNumberFormatter());

    TableColumn<SortStat, String> name = new TableColumn<>( s: "Name");
    name.setCellValueFactory( CellDataFeatures<SortStat, String> data -> data.getValue().algorithm);

    TableColumn<SortStat, Number> sizes = new TableColumn<>( s: "Name");
    sizes.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().size);

    TableColumn<SortStat, Number> comparisons = new TableColumn<>( s: "Comparisons");
    comparisons.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().comparisons);

    TableColumn<SortStat, Number> swaps = new TableColumn<>( s: "Swaps");
    swaps.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().swaps);

    TableColumn<SortStat, Number> writes = new TableColumn<>( s: "Writes");
    writes.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().writes);

    TableColumn<SortStat, Number> maxRuntime = new TableColumn<>( s: "Max Runtime");
    maxRuntime.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().maxRuntime);

    TableColumn<SortStat, Number> minRuntime = new TableColumn<>( s: "Min Runtime");
    minRuntime.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().minRuntime);

    TableColumn<SortStat, Number> meanRuntime = new TableColumn<>( s: "Mean Runtime");
    meanRuntime.setCellValueFactory( CellDataFeatures<SortStat, Number> data -> data.getValue().meanRuntime);

    algorithmStats.getColumns().addAll(name,sizes,comparisons, swaps, writes, minRuntime, maxRuntime, meanRuntime);

}
```

Figure 14: ComparisonController Pt1

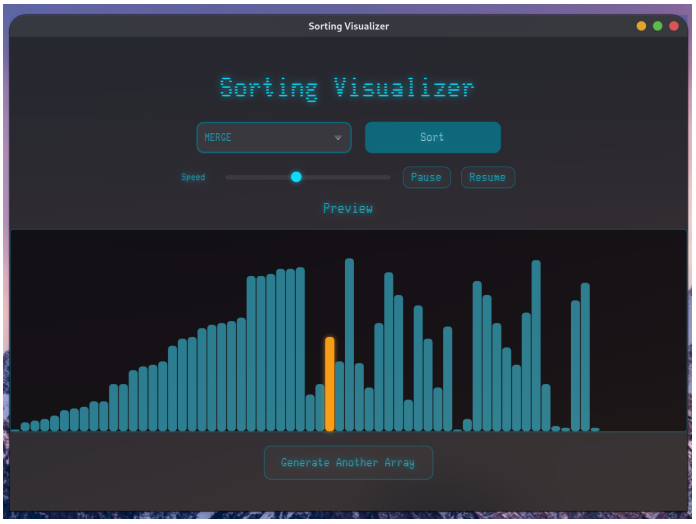
ComparisonController This is the initialization method of the controller, it adds the needed columns to the TableView linking to the FXML, defining the data captured by each one SetCellViewFactory() method.

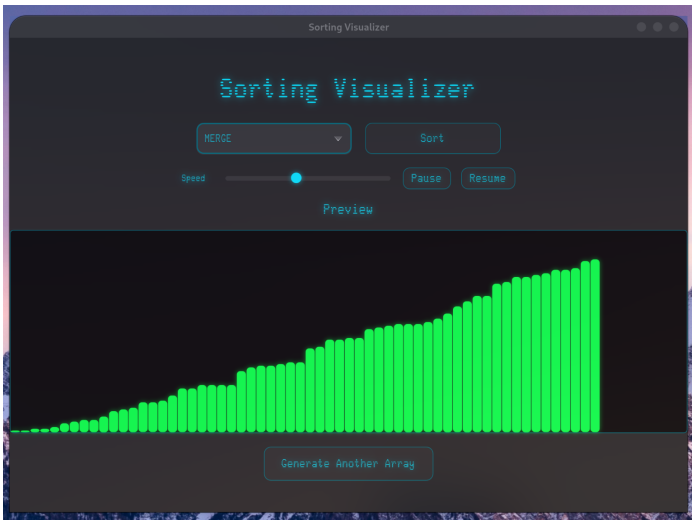
```java
@FXML  nelatawy
private void runComparison(ActionEvent actionEvent) {
    Thread sortingThread = new Thread(()->{
        System.out.println(runCount.getText());
        for (Node node : algorithmCheckboxes.getChildren()){
                addAlgorithmStats((CheckBox)  node);

        }
    });
    sortingThread.start();
}

private void addAlgorithmStats(CheckBox checkBox) {  1 usage  nelatawy *
    if(!checkBox.isSelected())
        return;
    int runs = Integer.parseInt(runCount.getText());

    long comparisonCnt = 0;
    long swapCnt = 0;
    long setCnt = 0;
    double maxRuntime = 0L;
    double minRuntime = 1e9;
    double totalRuntime = 0L;
    double meanRuntime = 0L;

    HBox algoStats = new HBox();
    algoStats.getChildren().add(new Label(checkBox.getText()));

    SortingStrategy<Integer> strategy = CommonUtils.getSortingAlgorithm(SortingAlgorithm.valueOf(checkBox.getText()));

    for (int i = 0; i < runs; i++) {
        long start = System.nanoTime();
        SortingMetrics<Integer> metrics = new SortingMetrics<>();
        strategy.sort(new ArrayList<>(arr), Comparator.naturalOrder(), metrics);
        long end = System.nanoTime();
        double runtime = (end - start)/1e6;

        comparisonCnt += metrics.getComparisonCount();
        swapCnt += metrics.getSwapCount();
        setCnt += metrics.getSetCount();

        totalRuntime += runtime;
        minRuntime = Math.min(minRuntime, runtime);
        maxRuntime = Math.max(maxRuntime, runtime);

    }
    meanRuntime = totalRuntime / runs;

    algorithmStats.getItems().add(
            new SortStat(
                    checkBox.getText(),
                    arr.size(),
                     comparisons: comparisonCnt/runs,  swaps: swapCnt/runs,  writes: setCnt/runs,
                    minRuntime, maxRuntime, meanRuntime
            ));
}
```

Figure 15: ComparisonController Pt2

And there is the core logic of the controller, it runs the comparison on all the selected algorithms and for each algorithm it runs the number of times specified, and it keeps track of the min/max runtimes and the average of all runs, the average and total number of swaps,comparisons and writes and adds them to the TableView to be shown.

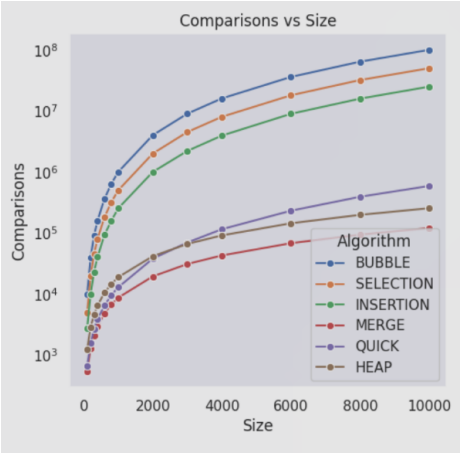
2 GUI Snapshots

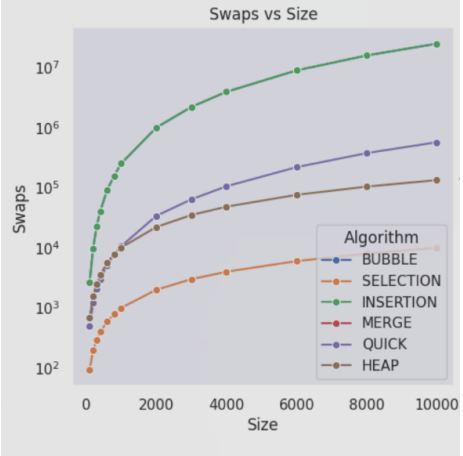


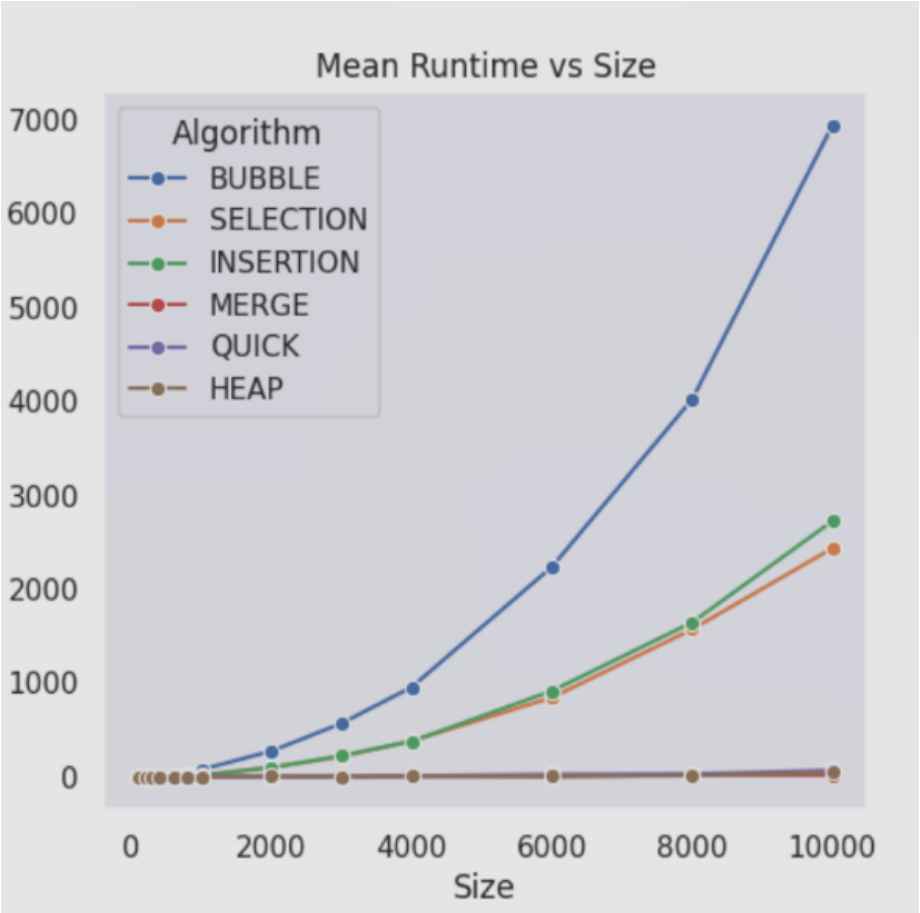

Sorting Visualizer
Sorting Visualizer
MERGE
Sort
Speed
Pause
Resume
Preview
Generate Another Array


Sorting Visualizer
Algorithm Comparison
Algorithms
BUBBLE
SELECTION
INSERTION
MERGE
QUICK
HEAP
Runs
30
Start Comparison
Save Stats
Reset
Name Size Comparisons Swaps Writes Min Runtime Max Runtime Mean Runtime
BUBBLE 200 39900 9609 0 1.373681 14.723393 3.109238933333346
SELECTION 200 19900 195 0 1.00249 6.753329 1.5845765666666665
INSERTION 200 9808 9609 197 0.757785 2.661682 1.2474045999999996
MERGE 200 1292 0 1544 0.254082 0.788234 0.3422510000000014
QUICK 200 1537 1108 0 0.122015 0.636899 0.2031723666666671
HEAP 200 2827 1543 0 0.429765 8.890556 1.2005118666666668
Generate another ...

3 Graphs


Comparisons vs Size
Comparisons
Size
Algorithm
BUBBLE
SELECTION
INSERTION
MERGE
QUICK
HEAP


Swaps vs Size
Swaps
Size
Algorithm
BUBBLE
SELECTION
INSERTION
MERGE
QUICK
HEAP



as we can see in the runtime plot, the Bubble, Insertion, and Selection take significantly longer than the $O(nlgn)$ algorithms, and they take the shape of a Quadratic function however the constants associated with it is significantly larger in magnitude in the Bubble sort than in the Selection,Insertion Sorting algorithms. Note that : the Swap and comparison count against the size is plotted with log scale.